Google Summer of Code 2026 Proposal

Fine-Grained Storage for the DUNE Experiment

Submitted to the **CERN-HEP Software Foundation (HSF)**

Corresponding Project: **DUNE**

Mentors

Wanwei Wu
Argonne National Laboratory
wanwei.wu@anl.gov

Peter van Gemmeren
Argonne National Laboratory
gemmeren@anl.gov

Personal Details

Name: Srijan Roy

Student: Department of Electronics and Telecommunication Engineering, Jadavpur University

Email: srijanroysr10@gmail.com

Phone: +91 9330961453

GitHub: github.com/SrijanRoy123-github

Resume: Resume Link

Timezone: Indian Standard Time (UTC +5:30)

Contents

Abstract **1**

1. Project Overview **1**

2. Motivation **2**

3. Understanding of the Framework R&D Ecosystem and Existing Benchmark Capabilities **2**

4. Key Technical Observations and Proposed Implementation Direction **2**

4.1 Reader benchmark as the central implementation point 3

4.2 Chunk-level processing and repeated reader creation 5

4.3 Nested asynchronous execution and thread budgeting 6

4.4 Hardcoded benchmark configuration in the driver 8

4.5 Writer-side timing methodology 10

4.6 Result structures and richer benchmark metadata 12

5. Planned Code-Level Improvements **14**

6. Proposed Deliverables **15**

7. Work Plan **15**

Phase 1: Reproduction and Repository Mapping 16

Phase 2: Benchmark Methodology Improvements 16

Phase 3: Systematic Read-Side Characterization 16

Phase 4: Optimization Study 16

8. Timeline **17**

9. Risk Assessment and Mitigation **18**

10. Personal Information **19**

10.1 Communication 20

10.2 Post GSoC 20

11. Contributions **20**

12. References and Bibliography **21**

Abstract

This project focuses on understanding and improving read-side performance in the fine-grained storage model being developed for the DUNE experiment[1]. In this model, logically distinct sub-records can be written eagerly into independent containers without synchronization during writing, which improves write-side scalability but shifts complexity to the read path.

The work will build on the public Framework R&D ecosystem around Phlex , with the main experimental effort centered on the phlex-io-granularity-test repository which already contains a benchmark stand comparing multiple storage organizations, including Array of Structs (AOS) and Struct of Arrays (SOA) configurations, across several hierarchy levels and grouping strategies[2].

My goal is to

- Extend the existing benchmark into a more systematic and methodologically robust framework
- Focus on detailed and structured analysis of the read-side performance
- Identify the main sources of inefficiency in the current design
- Evaluate practical optimization strategies such as reader reuse and caching

The expected result is a reproducible benchmark workflow, a quantitative understanding of read-side trade-offs across representative layouts, and a prototype optimization study that can inform the future i/o framework.

1. Project Overview

The Deep Underground Neutrino Experiment (DUNE) requires software infrastructure that can handle large, structured detector data efficiently[3]. To support growing record size and more flexible workflows, the framework effort around Phlex [4]is exploring a fine-grained storage model in which logically distinct sub-records are written into independent containers with minimal synchronization during writing.

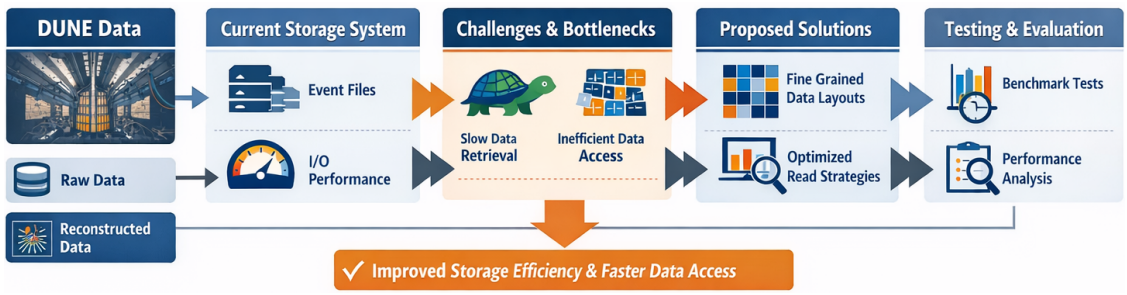

Figure 1: High-level view of the fine-grained storage problem: write-side independence improves scalability, while the read path must reconstruct related data across independent containers.

The main experimental basis for this project is the phlex-io-granularity-test repository [2], which already benchmarks multiple storage organizations, including Array-of-Structs (AOS) and Struct-of-Arrays (SOA), across event-, spill-, top-object-, and element-level layouts. This project will build on that benchmark to make the study more systematic, improve the measurement methodology, and evaluate practical read-side optimizations.

Core project objective: Systematically characterize and improve read-side performance when logically related data is stored across multiple independent containers without synchronization at write time.

2. Motivation

Phlex is intended for workflows where data is not always best represented by a single rigid event hierarchy[4, 5]. That flexibility makes it a good match for DUNE, but it also means that related pieces of information may be stored separately rather than as one tightly coupled unit.

This is helpful during writing because it reduces coordination and can make output more scalable. The difficulty appears later: reading back a full logical record may require fetching several separate pieces, re-establishing their relationships, and coordinating access across containers.

That trade-off is the real motivation for this project. The important question is not only whether fine-grained storage works, but whether it remains efficient for realistic reads. I therefore want to study which layout choices behave well, where the current approach becomes expensive, and which improvements can reduce that cost in a measurable way.

3. Understanding of the Framework R&D Ecosystem and Existing Benchmark Capabilities

The project sits within a broader Framework-R&D ecosystem[6], and understanding this ecosystem is important for doing meaningful work rather than isolated benchmarking. The major repositories include phlex[4], which is the core framework implementation; phlex-design[5], which documents the architecture and design rationale; phlex-examples[7], which provides minimal working workflows and user-facing execution examples; phlex-spack-recipes[8], which supports installation and packaging; phlex-coding-guidelines[9], which defines coding and testing standards; meld[10], which represents earlier exploratory work on flexible data organization; and phlex-io-granularity-test[2], which is the repository most directly relevant to this project.

Within phlex-io-granularity-test, the most important files for this project are HitWireReaders.hpp and HitWireReaders.cpp, which define the layout-specific read-side benchmark paths; HitWireWriters.hpp and HitWireWriters.cpp, which implement the corresponding write-side layouts; ReaderResult.hpp and WriterResult.hpp, which store benchmark results; main.cpp, which centralizes benchmark configuration and orchestrates the full benchmarking workflow; Utils.hpp and Utils.cpp, which provide shared utilities such as range splitting; and the tests folder, which validates important ROOT or RNTuple assumptions used by the benchmark. At the same time, the current benchmark already provides a strong foundation by comparing multiple layout classes and grouping strategies. It includes different persistent organizations such as event level, spill level, top object level, and element level layouts, combined with alL-data-product, per-data-product, and per-group placement strategies, for both AOS and SOA organizations where applicable.

4. Key Technical Observations and Proposed Implementation Direction

After studying the current implementation of the *phlex-io-granularity-test* benchmark repository [2], I identified a set of concrete implementation patterns that shape both the present benchmark behaviour and the most meaningful directions for improvement. These observations not only reveal limitations in the current code, but also indicate where the benchmark can be restructured so that its measurements become cleaner, more systematic, and more useful for the DUNE storage problem [1].

My goal is not simply to modify the benchmark implementation, but to evolve it into a stronger experimental framework whose outputs can be interpreted quantitatively. In particular, I want the benchmark to support clear evaluation using metrics such as:

- cold-read latency

- warm-read latency
- read throughput under representative scenarios
- scaling behaviour with thread count
- file-size overhead across layouts
- and consistency of results across repeated runs.

These metrics are especially relevant because the current benchmark is built around ROOT RNTuple-based read and write paths [11, 12], and the underlying storage and traversal behaviour can depend strongly on layout choice, grouping strategy, and execution structure. The most relevant files for this work are the reader paths in `HitWireReaders.cpp`, the writer paths in `HitWireWriters.cpp`, the benchmark driver in `main.cpp`, and the result structures in `ReaderResult.hpp` and `WriterResult.hpp`. Together, these define how layouts are generated, how reads are scheduled, how timings are collected, and how benchmark outputs are summarized [2].

4.1 Reader benchmark as the central implementation point

The read-side benchmark is the central implementation point for this project because the main question is how storage layout affects read performance. The current reader implementation already provides a useful foundation: it distinguishes cold and warm runs, compares multiple layout-specific access paths, and traverses data using cluster-based chunking. This means the repository already exposes several of the quantities that matter most for the project, especially cold-read latency, warm-read behaviour, and repeated-access cost.

At the same time, the present structure still mixes together several concerns that would ideally be separated:

- reader creation
- chunk scheduling
- traversal of deserialized objects
- and benchmark timing and metric collection

This makes it harder to answer an important methodological question: when the benchmark reports a slowdown, how much of that cost comes from the storage layout itself, and how much comes from the current implementation structure?

To address this, I would like to reorganize the reader code so that these responsibilities become more explicit. Instead of having one path that performs everything at once, I plan to move toward a structure with:

- a **reader-session layer** responsible for opening and owning the reader state
- a **read-plan or scheduling layer** responsible for partitioning work
- a **traversal layer** responsible only for visiting data products
- and a **metrics layer** responsible for timing, aggregation, and benchmark bookkeeping.

This separation is important because it would make the benchmark much easier to interpret. For example, it would allow the same logical workload to be executed under different reader strategies while preserving comparable metrics such as latency, throughput, and scaling behaviour. It would also

make it easier to validate that repeated-run behaviour is statistically consistent and not dominated by accidental implementation overhead.

A simplified structural sketch of the intended organization is shown below.

```
struct ReaderConfig {
    //string filenme, tplenme, fldnme;
    //int nThreads, iterations;
};
struct ReaderSession {
    ReaderConfig cfg;
    ReaderSession(const ReaderConfig& c) : cfg(c) {}
    void open() {
        // we have to open the ntuple reader and prepare any reusable reader state
    }
    void close() {
        // then release reader resources and clean up state
    }
};
struct ReadPlan {
    vector<pair<size_t, size_t>> chunks;
    static ReadPlan from_clusters(const ReaderSession& session, int nThreads) {
        ReadPlan plan;
        // to inspect cluster boundaries and divide work into chunk ranges
        return plan;
    }
};

struct ReaderMetrics {
    double cold = 0.0, warmAvg = 0.0, warmStddev = 0.0;
};
ReaderMetrics read_scenario(const ReaderConfig& cfg) {
    ReaderSession session(cfg);
    session.open();
    ReadPlan plan = ReadPlan::from_clusters(session, cfg.nThreads);
    ReaderMetrics metrics;

    // run the first pass as the cold read
    // run the remaining passes as warm reads
    // execute traversal over the planned chunks
    // collect timings and summarize warm-read statistics

    session.close();
    return metrics;
```

Figure 2: Proposed high-level reader structure. This organization separates reader-session creation, read-plan construction, chunk execution, and metric collection, making alternative read strategies easier to compare under consistent benchmark metrics.

With this structure, the reader benchmark can evolve from a useful prototype into a more controlled evaluation framework, one that not only reports timings but also supports stronger conclusions about which layouts scale well, which scenarios suffer the largest read-side cost, and which optimizations produce measurable improvement over the baseline.

4.2 Chunk-level processing and repeated reader creation

```cpp
void processNtupleRange(const std::string& fileName, const std::string& ntupleName, const std::string& fieldName, const std::pair<std::size_t, std::size_t>& chunk) {
    auto ntuple = ROOT::RNTupleReader::Open(ntupleName, fileName);
    auto view = ntuple->GetView<ViewType>(fieldName);
    for (std::size_t i = chunk.first; i < chunk.second; ++i) {
        const auto& val = view(i);
        traverse(val);
    }
}

template <typename ViewType>
double processNtuple(const std::string& fileName, const std::string& ntupleName, const std::string& fieldName, int nThreads) {
    auto pilot = ROOT::RNTupleReader::Open(ntupleName, fileName);
    auto chunks = Utils::split_range_by_clusters(*pilot, nThreads);
    std::vector<std::future<void>> futures;
    for (const auto& chunk : chunks) {
        futures.emplace_back(std::async(std::launch::async, processNtupleRange<ViewType>, fileName, ntupleName, fieldName, chunk));
    }
    for (auto& f : futures) f.get();
    return 0.0; // Placeholder, actual time measured outside
}
```

Figure 3: Chunk-level processing in `HitWireReaders.cpp`. The current implementation opens an `RNTupleReader` inside chunk processing and launches asynchronous tasks over cluster-based chunks.

The code shown above highlights one of the most important reader-side choices in the current implementation: chunk processing is directly tied to repeated reader opening. This is methodologically significant because the benchmark may therefore be measuring more than traversal cost alone. In particular, the reported read time may include a nontrivial contribution from repeated reader construction, initialization, and scheduling overhead.

For some scenarios, this may be a meaningful part of the end-to-end cost and should remain visible. However, for other scenarios, it can obscure the effect we actually want to study, namely how storage layout influences read-side performance. If these two sources of overhead are not separated, it becomes harder to interpret observed changes in cold-read latency, warm-read latency, and scaling behaviour.

For that reason, one of my planned improvements is to separate this behaviour into two explicitly defined benchmark modes:

- a **fresh-open mode**: where reader creation remains part of the measured path and contributes to the reported runtime
- a **reused-reader mode**: where reader state is created in a controlled way and reused across chunk traversal so that traversal and scheduling effects can be studied more directly.

This distinction is important because it enables a much more informative comparison. Instead of treating the current implementation as a single monolithic read path, the benchmark can separately report:

- the end-to-end latency of the current baseline behaviour
- the latency of a reused-reader traversal path
- the relative improvement between the two
- and the scenarios in which repeated opening dominates the total cost.

The intended structure would look like:

```
ReaderSession session(fileName, ntupleName, fieldName);
session.open();
// for the reused-reader path, we can proceed like this
for (auto chunk : assignedChunks) {
    session.traverse(chunk);
}
//and if we do the baseline part.....
for (auto chunk : assignedChunks) {
    // open a fresh reader for this chunk
    // traverse only this chunk
    // destroy the reader after traversal
}
```

Figure 4: Proposed comparison between a reused-reader path and the current baseline repeated-open path. The reused-reader structure isolates traversal cost more clearly, while the baseline preserves reader creation inside the measured chunk-processing path.

By separating these two strategies explicitly, the benchmark can answer a much clearer question: how much of the current read cost comes from the storage layout itself, and how much comes from the way readers are currently created and scheduled? This also gives a concrete success criterion for the project, namely that the final benchmark should be able to quantify the relative contribution of reader-opening overhead to the measured baseline and determine whether a reused-reader strategy produces a meaningful improvement in latency or throughput under representative read scenarios.

4.3 Nested asynchronous execution and thread budgeting

```
double readAOS_event_perGroup(const std::string& fileName, int nThreads) {
    TStopwatch sw;
    sw.Start();
    auto hitsFuture = std::async(std::launch::async, processNtuple<std::vector<HitIndividual>>, fileName, "aos_hits", "hits", nThreads);
    auto wiresFuture = std::async(std::launch::async, processNtuple<std::vector<WireBase>>, fileName, "aos_wires", "wires", nThreads);
    auto roisFuture = std::async(std::launch::async, processNtuple<std::vector<FlatROI>>, fileName, "aos_rois", "rois", nThreads);
    hitsFuture.get();
    wiresFuture.get();
    roisFuture.get();
    sw.Stop();
    return sw.RealTime();
}
```

Figure 5: Representative layout-specific reader function in `HitWireReaders.cpp`. Product-level futures are launched for hits, wires, and ROIs, while each of those paths may also trigger chunk-level parallelism.

A second important issue in the current reader benchmark is how parallelism is organized. The layout-specific readers already exploit concurrency, but they do so in a nested way: one reader may launch futures for multiple products such as hits, wires, and ROIs, while each of those product-level paths may further launch chunk-level asynchronous work. This creates a methodological problem, because the benchmark may then reflect not only storage-layout behaviour, but also oversubscription and scheduling overhead.

This matters directly for evaluation. If thread creation and task nesting are not controlled, then changes in latency, throughput, or scaling behaviour may be caused by the scheduling structure rather than by the layout being studied. In such a case, it becomes difficult to interpret whether one

configuration is genuinely better for reading or whether it simply interacts more favourably with the current parallel execution pattern.

To address this, I plan to replace implicit nested expansion with a more explicit read-plan structure. Instead of letting layout-specific readers recursively generate parallel work, the benchmark would first construct a read plan and then execute it using a controlled thread budget.

```
struct ReadPlan {
    string layout;
    vector<string> products;
    vector<pair<size_t, size_t>> chunks;
    int threadBudget;
};

ReadPlan build_read_plan(string layout,
                         vector<string> products,
                         int threadBudget) {
    ReadPlan plan;
    plan.layout = layout;
    plan.products = products;
    plan.threadBudget = threadBudget;

    // we have to choose how the work should be split
    // decide ehich is better: chunk-level or product-level parallelism
    // prepare chunk ranges to align with the system

    return plan;
}

struct Executor {
    ReadPlan plan;

    Executor(const ReadPlan& p) : plan(p) {}

    void run() {
        // to run the benchmark using the chosen scheduling strategy
        // keep thread usage controlled and comparable across scenarios
    }
};

auto plan = build_read_plan(layout, products, threadBudget);
Executor exec(plan);
exec.run();
```

Figure 6: Proposed read-plan and executor structure. The idea is to make scheduling decisions explicit by first constructing a read plan and then executing it with a controlled parallel strategy.

In practice, this means:

- deciding whether parallelism should occur primarily at the **product level** or the **chunk level**
- preventing uncontrolled nesting of asynchronous tasks
- and making thread allocation explicit and comparable across benchmark scenarios.

A successful outcome here would be a reader benchmark whose scaling behaviour is easier to interpret and more statistically consistent across repeated runs. More specifically, the improved structure should make it possible to compare layouts under a fixed thread budget and determine whether the observed read-side cost comes from storage fragmentation or from the execution strategy itself. This would strengthen both the reliability of the baseline and the validity of any later optimization study.

4.4 Hardcoded benchmark configuration in the driver

```cpp
int main() {
    ROOT::EnableThreadSafety();
    int nThreads = std::thread::hardware_concurrency();
    //int nThreads = 1;
    ROOT::EnableImplicitMT(nThreads);
    gSystem->Load("libWireDict");

    // Configuration parameters
    int numEvents = 1000;
    int hitsPerEvent = 100;
    int wiresPerEvent = 100;
    int roisPerWire = 10;
    int numSpills = 10;
    const std::string kOutputDir = "./output";

    // Create output directory if it doesn't exist
    std::filesystem::create_directories(kOutputDir);

    // Commented: AOS writer/reader benchmarks
    auto aos_writer_results = outAOS(nThreads, 3, numEvents, hitsPerEvent, wiresPerEvent, roisPerWire, numSpills, kOutputDir);
    visualize_aos_writer_results(aos_writer_results);
    auto aos_reader_results = inAOS(nThreads, 3, kOutputDir);
    visualize_aos_reader_results(aos_reader_results);

    // Collect AOS file sizes
    std::vector<std::pair<std::string, double>> aos_file_sizes;
    std::vector<std::string> aos_files = {
        kOutputDir + "/aos_event_all.root",
        kOutputDir + "/aos_event_perData.root",
        kOutputDir + "/aos_event_perGroup.root",
        kOutputDir + "/aos_spill_all.root",
        kOutputDir + "/aos_spill_perData.root",
        kOutputDir + "/aos_spill_perGroup.root",
        kOutputDir + "/aos_topObject_perData.root",
        kOutputDir + "/aos_topObject_perGroup.root",
        kOutputDir + "/aos_element_perData.root",
        kOutputDir + "/aos_element_perGroup.root"
    };
    for (const auto& f : aos_files) {
        auto tfile = TFile::Open(f.c_str(), "READ");
        if (tfile) {
```

Figure 7: Current hardcoded benchmark parameters in `main.cpp`.

The benchmark driver in `main.cpp` currently keeps all major benchmark parameters directly in code, including the number of events, hits per event, wires per event, ROIs per wire, spills, thread count, and output settings. This is reasonable for an initial prototype, but it makes controlled parameter sweeps and reproducible scenario comparison more difficult than necessary.

This matters because one of the main goals of the project is not only to measure a few selected cases, but to compare layouts systematically under clearly defined workloads. If benchmark settings are embedded as local variables in the driver, then reproducing a specific result or varying one parameter at a time becomes more error-prone and less transparent. This weakens both the comparability of results and the clarity of validation.

To address this, I plan to replace the current hardcoded setup with a more explicit benchmark-configuration structure. Instead of treating the benchmark as a single fixed run, the driver would take a configuration object describing the experiment and a scenario object describing the storage layout being tested.

- dataset shape
- thread count
- iteration count
- output location

- selected layout family
- hierarchy level
- and split strategy.

A proposed structure is shown below.

```
struct BenchmarkConfig {
    int numEvents;
    int hitsPerEvent;
    int wiresPerEvent;
    int roisPerWire;
    int numSpills;
    int nThreads;
    int iterations;
    string outputDir;
};

struct Scenario {
    string layout;         // for AOS or SOA elements
    string hierarchy;      // regarding the event,spill,topObject or element variables
    string splitStrategy;  // for the all,perData and perGroup functioms
};

void run_benchmark(const BenchmarkConfig& cfg, const Scenario& sc) {
    // generate or select the required benchmark setup
    // run writer benchmark for this scenario
    // run reader benchmark for this scenario
    // collect timings, scaling data, and output summaries
}
```

Figure 8: Proposed benchmark-configuration structure. Separating benchmark settings from layout scenarios would make the driver easier to extend, easier to reproduce, and better suited for controlled comparison across storage layouts.

This change would improve the benchmark in three ways. First, it would make experiments easier to reproduce by ensuring that every run is described by explicit parameters rather than scattered local variables. Second, it would support more systematic evaluation of success criteria such as latency, throughput, and scaling behaviour across multiple layouts. Third, it would make the benchmark easier to extend later, since new scenarios could be added without restructuring the driver each time.

A successful outcome here would be a benchmark driver that can run a clearly defined set of scenarios under controlled configurations and produce results that are easier to compare, validate, and repeat across multiple runs.

4.5 Writer-side timing methodology

```cpp
// Move executeInParallel to the top of the file, before any function implementations
static double executeInParallel(int totalEvents, int nThreads, const std::function<double(int, int, unsigned, int)>& workFunc) {
    if (nThreads <= 0 || totalEvents < 0) return 0.0;
    if (totalEvents == 0) return 0.0;
    auto seeds = Utils::generateSeeds(nThreads);
    int chunk = totalEvents / nThreads;
    std::vector<std::future<double>> futures;
    for (int th = 0; th < nThreads; ++th) {
        int start = th * chunk;
        int end = (th == nThreads - 1) ? totalEvents : start + chunk;
        if (start >= end) continue;
        futures.push_back(std::async(std::launch::async, workFunc, start, end, seeds[th], th));
    }
    double totalTime = 0.0;
    for (auto& f : futures) {
        totalTime += f.get();
    }
    return totalTime;
}
```

Figure 9: Parallel execution helper in `HitWireWriters.cpp`.

The writer benchmark is already useful, but its timing methodology deserves closer review. In particular, the parallel execution helper currently aggregates worker-returned values, while the reader benchmark measures elapsed time using stopwatch-style wall-clock timing. This creates an important methodological issue: the writer-side and reader-side benchmarks may not be reporting the same notion of runtime, which makes direct comparison less clear than it should be.

This matters because one of the project goals is to compare layout trade-offs across both writing and reading. If writer timings mostly reflect accumulated worker-local computation while reader timings reflect end-to-end elapsed latency, then apparent differences between layouts may partly come from how time is measured rather than from the storage design itself. To interpret the benchmark correctly, these two notions of runtime should be separated explicitly.

For that reason, I plan to distinguish:

- **elapsed benchmark time**, i.e. the wall-clock time observed by the benchmark user
- **worker-local computation time**, i.e. timing accumulated inside individual worker tasks.

A proposed timing-summary structure is shown below.

```
struct TimingSummary {
    double elapsed = 0.0;
    vector<double> workerTimes;
};
TimingSummary measure_parallel_run() {
    TimingSummary summary;
    // measure total wall-clock benchmark time
    // summary.elapsed = ...
    // collect worker-local timings separately if needed
    // summary.workerTimes = ...

    return summary;
}
```

Figure 10: Proposed timing-summary structure. The improved benchmark would separate total elapsed time from worker-local timings so that writer-side and reader-side measurements can be interpreted more clearly.

This change would strengthen the benchmark in two ways. First, it would make writer-side results easier to interpret on their own, since the reported numbers would clearly distinguish user-visible runtime from internal worker timing. Second, it would make comparison with the reader benchmark more consistent, especially when evaluating layout-dependent behaviour such as latency, throughput, and scaling with thread count.

A successful outcome here would be a writer benchmark that reports timings in a clearer and more reproducible way, so that any comparison between write-side and read-side behaviour is based on explicitly defined metrics rather than mixed timing semantics.

4.6 Result structures and richer benchmark metadata

```cpp
#ifndef READERRESULT_HPP
#define READERRESULT_HPP

#include <string>
#include <vector>

struct ReaderResult {
    std::string label;
    double cold;
    double warmAvg;
    double warmStddev;
    std::vector<double> coldTimes; // Store individual cold iteration times
    std::vector<double> warmTimes; // Store individual warm iteration times
    bool failed = false;
    std::string errorMessage = "";
};

std::vector<ReaderResult> in(int nThreads, int iter);

#endif
```

```cpp
#ifndef WRITERRESULT_HPP
#define WRITERRESULT_HPP

#include <string>
#include <vector>

struct WriterResult {
    std::string label;
    double avg;
    double stddev;
    std::vector<double> iterationTimes; // Store individual iteration times
    bool failed = false;
    std::string errorMessage = "";
};

#endif
```

Figure 11: Current result structures in `ReaderResult.hpp` and `WriterResult.hpp`.

The current result structures already provide a useful starting point: the benchmark records cold and warm read timings, iteration-level measurements, writer averages, and standard deviations. However, these structures still behave mostly as timing containers. For a benchmark that compares many layouts and scenarios, I believe the result objects should also serve as *scenario descriptors*, so that each reported measurement is tied explicitly to the conditions under which it was produced.

This is important for both interpretation and validation. If results are stored only as labels and summary timings, then later comparison becomes harder, especially when trying to analyze scaling behaviour, repeated-run consistency, or layout-dependent effects across several benchmark scenarios. Richer metadata would make the benchmark easier to reproduce, easier to validate against baseline

runs, and easier to extend as new scenarios are added.

For that reason, I would like to evolve the current result structures so that they record not only metrics, but also the main properties of the benchmark scenario:

- layout family (AOS or SOA)
- hierarchy level (event, spill, top object, or element)
- split strategy
- thread count
- iteration count
- selected workload-size parameters where relevant.

Conceptually, the improved structure would move toward a separation between **scenario description**, **measured metrics**, and **failure or validation state**. This would make each result more self-describing and would allow plots, tables, and statistical comparisons to be generated in a more systematic way.

A successful outcome here would be a benchmark result format that makes downstream analysis much more robust. In practice, this means that repeated runs, baseline comparisons, scaling studies, and optimization results can all be compared using explicit metadata rather than relying only on manually interpreted labels. This would strengthen both the readability of the benchmark outputs and the reproducibility of the final conclusions.

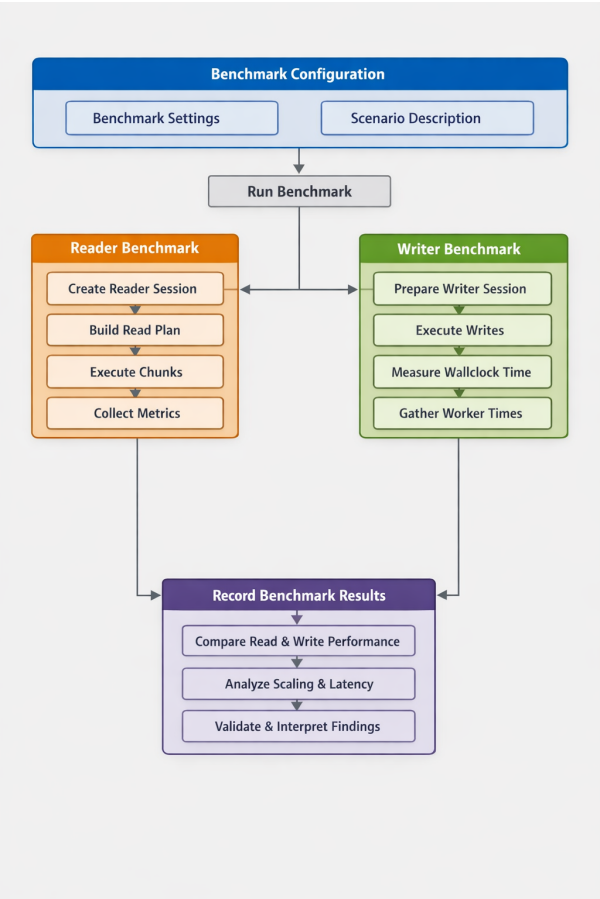

Figure 12: End-to-end benchmark workflow. The process begins with explicit benchmark configuration and scenario definition, followed by controlled execution of reader and writer benchmarks. Results are then recorded, compared, and analyzed to evaluate latency, throughput, and scaling behaviour across different storage layouts.

5. Planned Code-Level Improvements

The technical observations above motivate a set of focused code-level improvements aimed at turning the current benchmark into a more controlled, reproducible, and analysis-friendly evaluation framework. My goal is not to redesign the repository unnecessarily, but to improve the parts that most directly affect interpretability of the results and the reliability of the conclusions drawn from them.

These improvements are intended to make the benchmark better at isolating storage-layout effects from implementation overhead, to support clearer measurement of latency, throughput, and scaling behaviour, and to make optimization studies easier to evaluate against a stable baseline.

The main code-level improvements I plan to make are the following:

1. **Separate responsibilities inside the reader benchmark.** The current reader path combines reader creation, scheduling, traversal, and metric collection too closely. I plan to make these layers more explicit so that different read strategies can be compared under the same logical workload and with more interpretable metrics.

2. **Introduce an explicit reader-reuse path in addition to the current repeated-open baseline.** This will allow direct comparison between two benchmark modes: one that preserves the current end-to-end behaviour and one that isolates traversal cost more clearly. The resulting benchmark should quantify how much of the baseline read latency comes from repeated reader creation.

3. **Replace implicit nested parallelism with an explicit read-plan execution model.** Instead of allowing product-level and chunk-level futures to expand recursively, I plan to make scheduling decisions explicit through a read-plan structure with a controlled thread budget. This should produce cleaner scaling behaviour and reduce ambiguity in interpreting benchmark slowdowns.

4. **Move the benchmark driver toward explicit configuration objects.** The current hard-coded setup in `main.cpp` is useful for a prototype, but not ideal for reproducible scenario sweeps. I want the benchmark to describe its dataset shape, runtime settings, and storage scenario through explicit configuration objects so that experiments can be repeated and varied more systematically.

5. **Clarify timing semantics on the writer side.** The benchmark should distinguish user-visible elapsed time from worker-local computation time wherever both are relevant. This will make writer-side results easier to interpret and improve consistency when comparing write-side and read-side behaviour.

6. **Extend benchmark results into self-describing scenario records.** I want result objects to include not only timings, but also the scenario metadata needed for reproducibility and downstream comparison. This includes layout family, hierarchy level, split strategy, thread count, iteration count, and selected workload parameters.

7. **Strengthen validation and comparison against baseline.** For each major change, I want the modified benchmark to preserve a clear baseline path so that improvements can be evaluated directly. This is important because the project should not only produce new code, but also demonstrate that the code yields more reliable measurements or better performance under repeated and controlled runs.

Overall, success for this part of the project means that the benchmark becomes easier to reproduce, easier to extend, and more trustworthy as an experimental tool. In practical terms, that means clearer measurement semantics, more explicit scenario definition, stronger baseline comparison, and a better foundation for evaluating layout-dependent read-side optimizations.

6. Proposed Deliverables

Core Deliverables

- **Reproducible benchmark workflow.** A cleaned and structured benchmark pipeline that supports controlled execution across multiple scenarios, with clearly defined configuration, execution, and result collection stages, built on the existing *phlex-io-granularity-test* infrastructure [2].
- **Baseline characterization of read-side behaviour.** Systematic measurement of cold-read latency, warm-read latency, throughput, and scaling behaviour across AOS and SOA layouts at different hierarchy levels and grouping strategies, aligned with the storage questions described in the HSF project statement [1].
- **Improved benchmark structure.** Refactored reader path with explicit separation of reader-session, scheduling, traversal, and metrics layers, together with a configurable benchmark driver for more reproducible parameter sweeps, within the current Framework-R&D benchmark repository [2].
- **Structured result and metadata reporting.** Result objects extended to include scenario metadata such as layout family, hierarchy level, split strategy, thread count, and workload parameters, enabling more systematic comparison and downstream analysis [2].
- **Validated baseline results.** Benchmark outputs that are reproducible across repeated runs, with explicit reporting of averages, variability, and consistency so that comparisons remain statistically meaningful. This is especially important given the ROOT RNTuple-based read and write paths used by the benchmark [11, 12].
- **Bottleneck analysis of read-side performance.** A clear breakdown of dominant cost components, including effects of layout choice, reader creation overhead, scheduling behaviour, and access patterns, for the DUNE fine-grained storage problem [1].
- **Prototype optimization and evaluation.** Implementation of at least one optimization strategy, such as reader reuse or caching, with direct comparison against the baseline and reporting of relative improvements in latency, throughput, and scaling behaviour.
- **Final documentation and technical summary.** A concise report describing methodology, benchmark design, results, and conclusions, together with documentation for reproducing experiments and understanding the benchmark structure [2, 4].

Stretch Goals (if time permits)

- Extended parameter sweeps across workload size, thread count, and layout combinations to study broader scaling trends.
- Evaluation of additional optimization strategies beyond the primary prototype.
- Automated result aggregation and plotting pipeline for faster analysis and comparison.

7. Work Plan

The project is organized into four stages, each producing concrete and measurable outputs. The overall goal is to move from a reproducible baseline to a validated optimization through controlled benchmarking.

Phase 1: Reproduction and Repository Mapping

- Build and run the `phlex-io-granularity-test` benchmark.
- Map layout families to their implementation and storage organization.
- Study reader, writer, and driver code paths.
- Verify reproducibility of baseline results.

Outcome: Reproducible baseline benchmark with clear mapping between layouts, implementation, and outputs.

Phase 2: Benchmark Methodology Improvements

- Refactor reader path into session, scheduling, traversal, and metrics layers.
- Add reader-reuse path alongside repeated-open baseline.
- Introduce controlled read-plan execution with explicit thread budgeting.
- Replace hardcoded parameters with configuration objects.
- Extend result structures with scenario metadata.

Outcome: Structured and reproducible benchmark framework with clear measurement semantics.

Phase 3: Systematic Read-Side Characterization

- Run experiments across layouts, hierarchy levels, and grouping strategies.
- Measure cold/warm latency, throughput, and scaling behaviour.
- Perform repeated runs to ensure statistical consistency.
- Compare results against baseline to identify bottlenecks.

Outcome: Validated benchmark results explaining layout-dependent performance.

Phase 4: Optimization Study

- Implement a primary optimization (e.g., reader reuse or caching).
- Compare optimized and baseline paths under identical conditions.
- Evaluate improvements in latency, throughput, and scaling.

Outcome: Prototype optimization with clear, reproducible performance evaluation.

This staged approach ensures that each phase produces measurable outputs and that final conclusions are supported by controlled and repeatable experiments.

8. Timeline

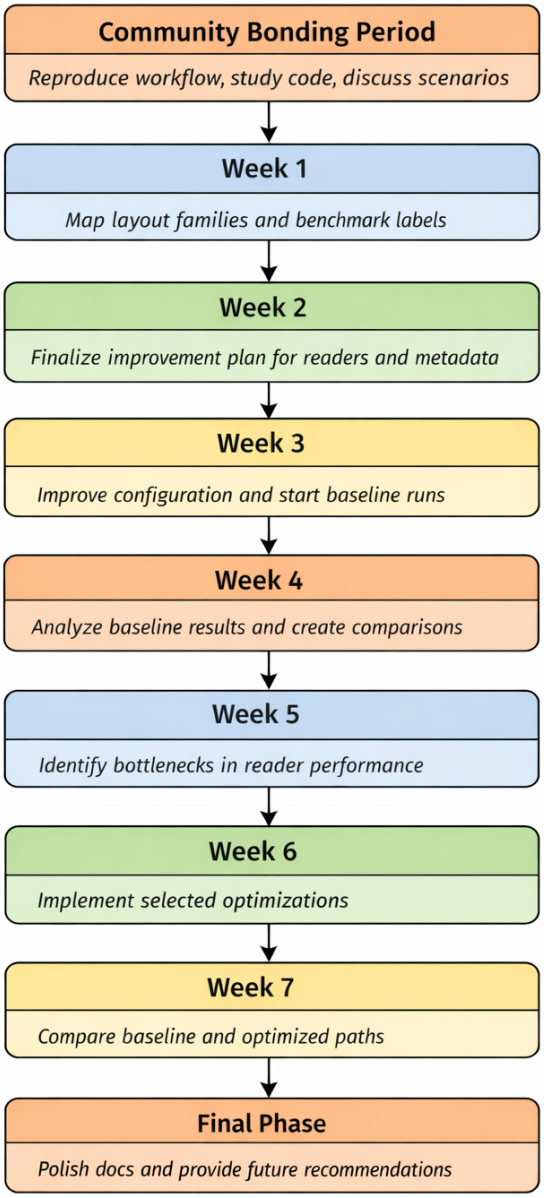

Figure 13: Flowchart representation of the proposed project timeline, from community bonding through final reporting and documentation.

Period	Planned Work	Checkpoint / Deliverable
Community Bonding Period	Build the benchmark repository and related repositories, reproduce the current workflow, study the main reader, writer, and driver paths, and finalize the initial benchmark scope with mentors.	**Working baseline setup** and repository map

Period	Planned Work	Checkpoints
Week 1	Document implemented layout families and map benchmark labels to storage organization, hierarchy level, and grouping strategy.	**Layout mapping note** and baseline scenario list
Week 2	Finalize the benchmark-improvement plan, especially for reader-side thread usage, repeated reader creation, and result metadata.	**Benchmark redesign plan**
Week 3	Implement benchmark-configuration cleanup and richer result structures; begin controlled baseline runs.	**Configurable driver** and updated result format
Week 4	Continue baseline experiments across selected layouts; organize early tables and plots for latency, throughput, and scaling.	**Initial plots** and baseline summary
Week 5	Analyze baseline results to identify dominant bottlenecks and validate consistency across repeated runs.	**Bottleneck analysis note** and validated baseline
Week 6	Implement the primary optimization path, such as reader reuse or a caching-related improvement.	**Prototype optimization** integrated into benchmark
Week 7	Run controlled comparisons between baseline and optimized paths under identical scenarios.	**Comparative performance results**
Week 8	Perform final validation runs, clean up benchmark code, and consolidate results into final plots and summaries.	**Final benchmark package** and result summary
Final Phase	Incorporate mentor feedback, polish documentation, and summarize conclusions and future directions.	**Final report** and cleaned documentation

9. Risk Assessment and Mitigation

Like most systems-oriented benchmarking projects, this work has a few practical risks. I summarize the main ones below together with the mitigation strategy I would follow.

- **Benchmark effects may be dominated by methodology rather than storage layout.** There is a risk that nested parallelism, repeated reader creation, or timing semantics may distort the layout-dependent effects I want to study. To mitigate this, I will preserve a clear baseline path, improve the benchmark structure in small steps, and validate each change against repeated runs before drawing conclusions.
- **Performance trends may vary strongly across scenarios.** Some optimizations may help only under specific workload shapes, hierarchy levels, or thread counts. To mitigate this, I will evaluate representative scenarios instead of relying on a single benchmark case, and I will report where an optimization is effective and where it is not.

- **Project scope may grow too broad for a 175-hour timeline.** Since the repository already supports multiple layouts and benchmark dimensions, there is a risk of trying to explore too many directions at once. To mitigate this, I will prioritize core deliverables first: reproducible baseline benchmarking, one structured bottleneck analysis, and one primary optimization study. Additional benchmark extensions will remain stretch goals.
- **Observed improvements may be small or inconclusive.** A proposed optimization such as reader reuse or caching may produce limited gains. This would still be a useful outcome if the benchmark clearly explains why the gain is limited. To mitigate this, I will focus not only on achieving speedup, but also on producing reproducible evidence about the dominant sources of cost.
- **Reproducibility and result consistency may be affected by runtime variability.** Benchmarking can be sensitive to repeated-run variation. To mitigate this, I will use repeated measurements, report averages and spread, and compare optimized paths directly against the same baseline conditions.

Overall, the main mitigation strategy is to keep the project staged, baseline-driven, and validation-focused. This ensures that even if some optimization directions are less effective than expected, the project still produces a reliable benchmark framework and a meaningful analysis of read-side performance trade-offs.

10. Personal Information

I am Srijan Roy, an undergraduate student in the Department of Electronics and Telecommunication Engineering at Jadavpur University, Kolkata. Over the last few years, I have developed a strong interest in research-oriented software development, especially at the intersection of computer vision, deep learning, scientific computing, and performance-aware systems. What attracts me most to open source is the chance to contribute to real technical problems, learn from existing codebases, and build work that is useful beyond a single project or classroom setting.

My current work is shaped by two main tracks. The first is research in computer vision and deep learning, where I have worked on spectral image analysis, semantic segmentation, remote sensing, deepfake detection, and video classification. This has involved building training and evaluation pipelines, performing benchmarking and cross-dataset analysis, and studying model robustness and deployment trade-offs. I have also co-authored research work on Spectral Representation Learning [13], and I maintain a Google Scholar profile listing my research output [14], with additional papers currently under submission and in preparation.

The second track is open-source and technical development work. I am currently a part-time contributor to the Formal Conjectures Project at Google DeepMind, where I work with Lean and formalized mathematics. I have also served as the Web Development Lead of the IoT Applications Club, Jadavpur University, where I worked on the design and development of the club website with the development team. The live club website is available at IOTA Website [15].

From the technical side, I am comfortable with C++, Python, C, JavaScript, HTML, CSS, Lean, and PHP. I regularly work with libraries and tools such as PyTorch, Torchvision, OpenCV, NumPy, scikit-learn, Albumentations, Matplotlib, CUDA, cuDNN, TensorRT, and NVIDIA JetPack SDK. I also have experience with Linux-based development, deployment platforms such as Vercel and Render, and web/database stacks involving MySQL and MariaDB.

More broadly, I enjoy projects that require both implementation and careful reasoning. My background in research has trained me to think in terms of experiments, baselines, comparisons, and measurable outcomes, while my software and web-development experience has helped me become comfortable with real codebases, iterative improvement, and collaborative engineering. I believe this

combination of research mindset and practical development experience is what makes me especially interested in a project like this one.

10.1 Communication

I value consistent communication and iterative feedback, especially in collaborative technical work. For a project like this one, where understanding existing code and interpreting benchmark results are both important, I believe regular discussion with mentors is essential. During the project period, I plan to share progress through concise updates, intermediate findings, and clear notes on any technical uncertainty or design decision that may benefit from early feedback.

I am based in India and work in Indian Standard Time (UTC +5:30). I am available to work full-time on weekdays, typically between 12 PM IST and 8 PM IST. However, I am flexible with my schedule and do not have strict time constraints, as I genuinely enjoy working on this project. I am comfortable with written communication over email, issue trackers, and repository discussions, and can adapt to the communication style preferred by the mentors. I am also open to adjusting my working hours whenever needed, making it easy to coordinate across time zones for discussions, reviews, or any collaborative work.

My general communication style is to keep mentors informed not only about what is completed, but also about what I am currently testing, what is not yet clear, and what trade-offs I am considering. Since this project involves benchmarking and interpretation, I believe that sharing partial observations early is often more useful than waiting until every detail is finalized. I also intend to maintain clear documentation of benchmark setups, assumptions, and implementation changes so that the work remains easy to review and reproduce.

10.2 Post GSoC

I would like this project to be the beginning of a longer engagement rather than a one-time summer contribution.The benchmark repository can serve as a reusable experimental test stand for future studies on layout design in the DUNE software ecosystem. For that reason, I would like to continue contributing after the formal GSoC period, especially if there are promising benchmark extensions or optimization directions that cannot be fully completed within the project timeline.

More broadly, I would like to remain involved with the Framework-R&D and HSF ecosystem, continue learning from the maintainers and mentors, and keep contributing where my skills are useful. Even if the initial GSoC work focuses on one well-scoped optimization study, I expect the results to naturally lead to follow-up questions about layout choice, benchmark methodology, and framework-level design decisions. I would be very interested in continuing that work and helping strengthen the benchmark and analysis infrastructure further.

I am especially motivated by projects that combine careful measurement, code understanding, and practical improvement. Because of that, I see this proposal not only as a summer project, but as a strong opportunity to contribute to a scientific software effort whose long-term impact goes well beyond the immediate deliverables.

11. Contributions

Here is the repository link of the evaluation task that i completed.

12. References and Bibliography

References

[1] HEP Software Foundation, *GSoC 2026 Project: Fine Grained Storage for the DUNE Experiment*, online. Available: https://hepsoftwarefoundation.org/gsoc/projects/2026/project_DUNE.html

[2] Framework-R&D, *phlex-io-granularity-test: Granularity test stand for storing Phlex type data product in RNTuple*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex-io-granularity-test

[3] Deep Underground Neutrino Experiment (DUNE), *Official DUNE Website*, online. Available: https://www.dunescience.org/

[4] Framework-R&D, *phlex: Parallel, Hierarchical, and Layered Execution of data-processing algorithms*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex

[5] Framework-R&D, *phlex-design: Design ideas for Phlex*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex-design

[6] Framework-R&D, *Framework-R&D GitHub Organization*, online. Available: https://github.com/Framework-R-D

[7] Framework-R&D, *phlex-examples: Examples for the use of the Phlex framework*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex-examples

[8] Framework-R&D, *phlex-spack-recipes: Repository for Spack recipes for Phlex and related dependencies*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex-spack-recipes

[9] Framework-R&D, *phlex-coding-guidelines: Coding guidelines for framework contributors*, GitHub repository, online. Available: https://github.com/Framework-R-D/phlex-coding-guidelines

[10] Framework-R&D, *meld: Exploratory work to meet DUNE's processing needs*, GitHub repository, online. Available: https://github.com/Framework-R-D/meld

[11] ROOT Project, *RNTuple Introduction*, online documentation. Available: https://root.cern/doc/v622/md_tree_ntuple_v7_doc_README.html

[12] ROOT Project, *ROOT::RNTuple Class Reference*, online documentation. Available: https://root.cern.ch/doc/v638/classROOT_1_1RNTuple.html

[13] B. Das, A. Ali, S. Roy, S. Anam, K. Djemal, and S. S. Chaudhuri, *Spectral Representation Learning in Color Space for Deforestation Mapping with Limited Supervision*, in *2025 IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 1192–1198, 2025. Available: https://doi.ieeecomputersociety.org/10.1109/ICTAI66417.2025.00174

[14] Srijan Roy, *Google Scholar Profile*, online. Available: https://scholar.google.com/citations?hl=en&user=SFW2ElkAAAAJ

[15] IoT Applications Club, Jadavpur University, *IOTA Website*, online. Available: https://www.iotaju.com/HOME/homepage-of-iota-main/home.html